

Final Project Covid

May 11, 2026

Analysis of Global COVID-19 Pandemic Data

Estimated time needed: **90** minutes

0.1 Overview:

In this final project, you will apply your knowledge of data analysis by completing a series of hands-on tasks in this lab notebook. The lab consists of 10 tasks, where you will write and execute code to demonstrate your skills.

While working through the lab, remember to save all your code and outputs, then download the Jupyter Notebook, as you will need to submit the completed notebook for the **Final Project: Submission and Evaluation**.

If you need to refresh your memories about specific coding details, you may refer to previous hands-on labs for code examples.

```
[ ]: # This lab requires 'httr' and 'rvest' packages, which are already pre-loaded
      ↪ into this lab environment.
      # However, if you are working on your local RStudio, please uncomment the below
      ↪ codes and install the packages.

      #install.packages("httr")
      #install.packages("rvest")
```

```
[1]: library(httr)
      library(rvest)
      print ("libraries are ready")
```

```
[1] "libraries are ready"
```

Note: if you can import above libraries, please use `install.packages()` to install them first.

0.2 TASK 1: Get a COVID-19 pandemic Wiki page using HTTP request

First, let's write a function to use HTTP request to get a public COVID-19 Wiki page.

Before you write the function, you can open this public page from this

URL https://en.wikipedia.org/w/index.php?title=Template:COVID-19_testing_by_country using a web browser.

The goal of task 1 is to get the html page using HTTP request (`httr` library)

```
[2]: get_wiki_covid19_page <- function() {

  # Our target COVID-19 wiki page URL is: https://en.wikipedia.org/w/index.php?
  ↪title=Template:COVID-19_testing_by_country
  # Which has two parts:
  # 1) base URL `https://en.wikipedia.org/w/index.php`
  # 2) URL parameter: `title=Template:COVID-19_testing_by_country`, seperated
  ↪by question mark ?

  # Wiki page base
  wiki_base_url <- "https://en.wikipedia.org/w/index.php"
  # You will need to create a List which has an element called `title` to
  ↪specify which page you want to get from Wiki
  # in our case, it will be `Template:COVID-19_testing_by_country`

  # - Use the `GET` function in httr library with a `url` argument and a
  ↪`query` argument to get a HTTP response

  # Use the `return` function to return the response

}
print ("got wiki")
```

```
[1] "got wiki"
```

Call the `get_wiki_covid19_page` function to get a http response with the target html page

```
[3]: # Call the get_wiki_covid19_page function and print the response
get_wiki_covid19_page <- function() {

  # Wiki page base
  wiki_base_url <- "https://en.wikipedia.org/w/index.php"

  # Query parameters
  query_params <- list(
    title = "Template:COVID-19_testing_by_country"
  )

  # Send GET request
  response <- httr::GET(url = wiki_base_url, query = query_params)

  # Return response
  return(response)
}

# Call the function and print the response
response <- get_wiki_covid19_page()
print(response)
```

```

Response [https://en.wikipedia.org/w/index.php?title=Template%3ACOVID-
19_testing_by_country]
  Date: 2026-05-11 17:56
  Status: 200
  Content-Type: text/html; charset=UTF-8
  Size: 457 kB
<!DOCTYPE html>
<html class="client-nojs vector-feature-language-in-header-enabled vector-fea...
<head>
<meta charset="UTF-8">
<title>Template:COVID-19 testing by country - Wikipedia</title>
<script>(function(){var className="client-js vector-feature-language-in-heade...
RLSTATE={"ext.globalCssJs.user.styles":"ready","site.styles":"ready","user.st...
<script>(RLQ=window.RLQ||[]).push(function(){mw.loader.impl(function(){return...
}};});});</script>
<link rel="stylesheet" href="/w/load.php?lang=en&modules=ext.cite.styles%...
...

```

0.3 TASK 2: Extract COVID-19 testing data table from the wiki HTML page

On the COVID-19 testing wiki page, you should see a data table <table> node contains COVID-19 testing data by country on the page:

Note the numbers you actually see on your page may be different from above because it is still an on-going pandemic when creating this notebook.

The goal of task 2 is to extract above data table and convert it into a data frame

Now use the `read_html` function in `rvest` library to get the root html node from response

```

[4]: # Get the root html node from the http response in task 1
      # Load required libraries
      library(httr)
      library(xml2)

      # Get the HTTP response
      response <- get_wiki_covid19_page()

      # Parse the response content and get the root HTML node
      root_html <- read_html(response)

      # Print the root HTML node
      print(root_html)

```

```

{html_document}
<html class="client-nojs vector-feature-language-in-header-enabled vector-
feature-language-in-main-menu-disabled vector-feature-language-in-main-page-
header-disabled vector-feature-page-tools-pinned-disabled vector-feature-toc-
pinned-clientpref-1 vector-feature-main-menu-pinned-disabled vector-feature-
limited-width-clientpref-1 vector-feature-limited-width-content-enabled vector-

```

```
feature-custom-font-size-clientpref-1 vector-feature-appearance-pinned-
clientpref-1 skin-theme-clientpref-day vector-sticky-header-enabled vector-toc-
available skin-theme-clientpref-thumb-standard" lang="en" dir="ltr">
```

```
[1] <head>\n<meta http-equiv="Content-Type" content="text/html; charset=UTF-8
```

```
...
```

```
[2] <body class="skin--responsive skin-vector skin-vector-search-vue mediawik
```

```
...
```

Get the tables in the HTML root node using `html_nodes` function.

```
[5]: # Get the table node from the root html node
```

```
# Load required library
```

```
library(rvest)
```

```
# Get the table node from the root HTML node
```

```
table_node <- html_node(root_html, "table")
```

```
# Print the table node
```

```
print(table_node)
```

```
{html_node}
```

```
<table class="box-Update plainlinks ombox ombox-content ambox-Update"
```

```
role="presentation">
```

```
[1] <tbody><tr>\n<td class="mbox-image"><span typeof="mw:File"><span><img alt
```

```
...
```

Read the specific table from the multiple tables in the `table_node` using the `html_table` function and convert it into dataframe using `as.data.frame`

Hint:- Please read the `table_node` with index 2(ex:- `table_node[2]`).

```
[6]: # Read the table node and convert it into a data frame, and print the data_
      ↪frame for review
```

```
# Read the table and convert it into a data frame
```

```
# covid_table <- html_table(table_node)
```

```
# FIX: extract all tables properly
```

```
tables <- html_table(root_html, fill = TRUE)
```

```
# original section *****
```

```
# Select correct table
```

```
# covid_table <- tables[[1]]
```

```
# covid_df <- as.data.frame(covid_table)
```

```
# covid_df <- as.data.frame(covid_table, stringsAsFactors = FALSE)
```

```
# Print the data frame
```

```
# print(covid_df)
```

```
# View result
```

```
# print(clean_covid_df)
```

```
# end of original section *****
```

```
# Check all table dimensions
```

```

for(i in seq_along(tables)) {
  cat("\nTable", i, "dimensions:", dim(tables[[i]]), "\n")
}

# Select the table with enough columns
covid_table <- NULL

for(i in seq_along(tables)) {
  if(ncol(tables[[i]]) >= 7) {
    covid_table <- tables[[i]]
    break
  }
}

# Convert to dataframe
covid_df <- as.data.frame(covid_table, stringsAsFactors = FALSE)

print(head(covid_df))

```

Table 1 dimensions: 1 2

Table 2 dimensions: 173 9

Table 3 dimensions: 1 2

Table 4 dimensions: 1 5

	Country or region	Date[a]	Tested	Units[b]	Confirmed(cases)
1	Afghanistan	17 Dec 2020	154,767	samples	49,621
2	Albania	18 Feb 2021	428,654	samples	96,838
3	Algeria	2 Nov 2020	230,553	samples	58,574
4	Andorra	23 Feb 2022	300,307	samples	37,958
5	Angola	2 Feb 2021	399,228	samples	20,981
6	Antigua and Barbuda	6 Mar 2021	15,268	samples	832
	Confirmed /tested,%	Tested /population,%	Confirmed /population,%	Ref.	
1	32.1	0.40	0.13	[1]	
2	22.6	15.0	3.4	[2]	
3	25.4	0.53	0.13	[3] [4]	
4	12.6	387	49.0	[5]	
5	5.3	1.3	0.067	[6]	
6	5.4	15.9	0.86	[7]	

0.4 TASK 3: Pre-process and export the extracted data frame

The goal of task 3 is to pre-process the extracted data frame from the previous step, and export it as a csv file

Let's get a summary of the data frame

```
[7]: # Print the summary of the data frame
summary(covid_df)
print ("task 3")
```

```
Country or region    Date[a]          Tested          Units[b]
Length:173          Length:173        Length:173        Length:173
Class :character     Class :character  Class :character  Class :character
Mode :character      Mode :character   Mode :character   Mode :character
Confirmed(cases)     Confirmed /tested,% Tested /population,%
Length:173          Length:173        Length:173
Class :character     Class :character  Class :character
Mode :character      Mode :character   Mode :character
Confirmed /population,% Ref.
Length:173          Length:173
Class :character     Class :character
Mode :character      Mode :character
```

```
[1] "task 3"
```

As you can see from the summary, the columns names are little bit different to understand and some column data types are not correct. For example, the **Tested** column shows as **character**.

As such, the data frame read from HTML table will need some pre-processing such as removing irrelevant columns, renaming columns, and convert columns into proper data types.

We have prepared a pre-processing function for you to conver the data frame but you can also try to write one by yourself

```
[8]: preprocess_covid_data_frame <- function(data_frame) {

  shape <- dim(data_frame)
  # Remove the World row
  # data_frame<-data_frame[!(data_frame$`Country.or.region`=="World"),]
  # Remove the last row
  # data_frame <- data_frame[1:172, ]
  # Remove World row if exists
  if ("Country.or.region" %in% colnames(data_frame)) {
    data_frame <- data_frame[!(data_frame$`Country.or.region` == "World"), ]
  }

  # Keep only first 7 columns (fix structural mismatch)
  data_frame <- data_frame[, 1:7]
  # We dont need the Units and Ref columns, so can be removed
  data_frame["Ref."] <- NULL
  data_frame["Units.b."] <- NULL
  # Renaming the columns
  names(data_frame) <- c("country", "date", "tested", "confirmed", "confirmed.
→tested.ratio", "tested.population.ratio", "confirmed.population.ratio")
  # Convert column data types
  data_frame$country <- as.factor(data_frame$country)
```

```

    data_frame$date <- as.factor(data_frame$date)
# original code replaced ***** :
#   data_frame$tested <- as.numeric(gsub(",", "", data_frame$tested))
#   data_frame$confirmed <- as.numeric(gsub(",", "", data_frame$confirmed))
#   data_frame$'confirmed.tested.ratio' <- as.
#     ↪ numeric(gsub(",", "", data_frame$`confirmed.tested.ratio`))
#   data_frame$'tested.population.ratio' <- as.
#     ↪ numeric(gsub(",", "", data_frame$`tested.population.ratio`))
#   data_frame$'confirmed.population.ratio' <- as.
#     ↪ numeric(gsub(",", "", data_frame$`confirmed.population.ratio`))
# end of original code replacement *****

clean_numeric <- function(x) {
  # Convert to character
  x <- as.character(x)
  # Remove commas
  x <- gsub(",", "", x)
  # Remove percentage signs
  x <- gsub("%", "", x)
  # Remove footnotes like [a], [1], etc.
  x <- gsub("\\[.*?\\]", "", x)
  # Remove all non-numeric characters except decimal point
  x <- gsub("[^0-9\\.]", "", x)
  # Convert empty strings to NA
  x[x == ""] <- NA
  return(as.numeric(x))
}

# Apply cleaning
data_frame$tested <- clean_numeric(data_frame$tested)
data_frame$confirmed <- clean_numeric(data_frame$confirmed)
data_frame$confirmed.tested.ratio <- clean_numeric(
  data_frame$confirmed.tested.ratio)
data_frame$tested.population.ratio <- clean_numeric(
  data_frame$tested.population.ratio)
data_frame$confirmed.population.ratio <- clean_numeric(
  data_frame$confirmed.population.ratio)

  return(data_frame)
}
print ("done")

```

```
[1] "done"
```

Call the preprocess_covid_data_frame function

```
[9]: # call `preprocess_covid_data_frame` function and assign it to a new data frame
clean_covid_df <- preprocess_covid_data_frame(covid_df)
```

```
print ("done")
```

```
Warning message in clean_numeric(data_frame$tested):  
"NAs introduced by coercion"  
Warning message in clean_numeric(data_frame$confirmed):  
"NAs introduced by coercion"  
Warning message in clean_numeric(data_frame$confirmed.tested.ratio):  
"NAs introduced by coercion"  
Warning message in clean_numeric(data_frame$tested.population.ratio):  
"NAs introduced by coercion"  
Warning message in clean_numeric(data_frame$confirmed.population.ratio):  
"NAs introduced by coercion"  
  
[1] "done"
```

Get the summary of the processed data frame again

```
[10]: # Print the summary of the processed data frame again  
print(clean_covid_df)
```

```
country  
1  
Afghanistan  
2  
Albania  
3  
Algeria  
4  
Andorra  
5  
Angola  
6  
Antigua and Barbuda  
7  
Argentina  
8  
Armenia  
9  
Australia  
10  
Austria  
11  
Azerbaijan  
12  
Bahamas  
13  
Bahrain  
14
```


Bangladesh
15
Barbados
16
Belarus
17
Belgium
18
Belize
19
Benin
20
Bhutan
21
Bolivia
22
Bosnia and Herzegovina
23
Botswana
24
Brazil
25
Brunei
26
Bulgaria
27
Burkina Faso
28
Burundi
29
Cambodia
30
Cameroon
31
Canada
32
Chad
33
Chile
34
China[c]
35
Colombia
36
Costa Rica
37
Croatia
38

Cuba
39
Cyprus[d]
40
Czechia
41
Denmark[e]
42
Djibouti
43
Dominica
44
Dominican Republic
45
DR Congo
46
Ecuador
47
Egypt
48
El Salvador
49
Equatorial Guinea
50
Estonia
51
Eswatini
52
Ethiopia
53
Faroe Islands
54
Fiji
55
Finland
56
France[f] [g]
57
Gabon
58
Gambia
59
Georgia[h]
60
Germany
61
Ghana
62

Greece
63
Greenland
64
Grenada
65
Guatemala
66
Guinea
67
Guinea-Bissau
68
Guyana
69
Haiti
70
Honduras
71
Hungary
72
Iceland
73
India
74
Indonesia
75
Iran
76
Iraq
77
Ireland
78
Israel
79
Italy
80
Ivory Coast
81
Jamaica
82
Japan
83
Jordan
84
Kazakhstan
85
Kenya
86

Kosovo
87
Kuwait
88
Kyrgyzstan
89
Laos
90
Latvia
91
Lebanon
92
Lesotho
93
Liberia
94
Libya
95
Lithuania
96
Luxembourg[i]
97
Madagascar
98
Malawi
99
Malaysia
100
Maldives
101
Mali
102
Malta
103
Mauritania
104
Mauritius
105
Mexico
106
Moldova[j]
107
Mongolia
108
Montenegro
109
Morocco
110

Mozambique
111
Myanmar
112
Namibia
113
Nepal
114
Netherlands
115
New Caledonia
116
New Zealand
117
Niger
118
Nigeria
119
North Korea
120
North Macedonia
121
Northern Cyprus[k]
122
Norway
123
Oman
124
Pakistan
125
Palestine
126
Panama
127
Papua New Guinea
128
Paraguay
129
Peru
130
Philippines
131
Poland
132
Portugal
133
Qatar
134

Romania
135
Russia
136
Rwanda
137
Saint Kitts and Nevis
138
Saint Lucia
139
Saint Vincent
140
San Marino
141
Saudi Arabia
142
Senegal
143
Serbia
144
Singapore
145
Slovakia
146
Slovenia
147
South Africa
148
South Korea
149
South Sudan
150
Spain
151
Sri Lanka
152
Sudan
153
Sweden
154
Switzerland[l]
155
Taiwan[m]
156
Tanzania
157
Thailand
158

Togo
 159
 Trinidad and Tobago
 160
 Tunisia
 161
 Turkey
 162
 Uganda
 163
 Ukraine
 164
 United Arab Emirates
 165
 United Kingdom
 166
 United States
 167
 Uruguay
 168
 Uzbekistan
 169
 Venezuela
 170
 Vietnam
 171
 Zambia
 172
 Zimbabwe
 173 .mw-parser-output .reflist-columns-2{column-width:30em}.mw-parser-output
 .reflist-columns-3{column-width:25em}body.skin-vector-2022 .mw-parser-output
 .reflist-columns-2{column-width:27em}body.skin-vector-2022 .mw-parser-output
 .reflist-columns-3{column-width:22.5em}.mw-parser-output .references[data-mw-
 group=upper-alpha]{list-style-type:upper-alpha}.mw-parser-output
 .references[data-mw-group=upper-roman]{list-style-type:upper-roman}.mw-parser-
 output .references[data-mw-group=lower-alpha]{list-style-type:lower-alpha}.mw-
 parser-output .references[data-mw-group=lower-greek]{list-style-type:lower-
 greek}.mw-parser-output .references[data-mw-group=lower-roman]{list-style-
 type:lower-roman}.mw-parser-output div.reflist-liststyle-upper-alpha
 .references{list-style-type:upper-alpha}.mw-parser-output div.reflist-liststyle-
 upper-roman .references{list-style-type:upper-roman}.mw-parser-output
 div.reflist-liststyle-lower-alpha .references{list-style-type:lower-alpha}.mw-
 parser-output div.reflist-liststyle-lower-greek .references{list-style-
 type:lower-greek}.mw-parser-output div.reflist-liststyle-lower-roman
 .references{list-style-type:lower-roman}\n^n^ Local time.\n\n^n^ For some countries
 it is unclear whether they report samples or cases. One person tested twice is
 recorded as one case and two samples.\n\n^n^ Excluding Taiwan.\n\n^n^ Excluding
 Northern Cyprus.\n\n^n^ Excluding Greenland and the Faroe Islands.\n\n^n^ Excluding

Overseas France.\n\n^ Testing data from 4 May to 12 May is missing because of the transition to the new reporting system SI-DEP.\n\n^ Excluding Abkhazia and South Ossetia.\n\n^ Data for residents only.\n\n^ Excluding Transnistria.\n\n^ Northern Cyprus is not recognized as a sovereign state by any country except Turkey.\n\n^ Includes data for Liechtenstein.\n\n^ Not a United Nations member.

date

1
17 Dec 2020
2
18 Feb 2021
3
2 Nov 2020
4
23 Feb 2022
5
2 Feb 2021
6
6 Mar 2021
7
16 Apr 2022
8
29 May 2022
9
9 Sep 2022
10
1 Feb 2023
11
11 May 2022
12
28 Nov 2022
13
3 Dec 2022
14
24 Jul 2021
15
14 Oct 2022
16
9 May 2022
17
24 Jan 2023
18
8 Jun 2022
19
4 May 2021
20
28 Feb 2022
21
5 Jun 2022

22
27 Sep 2022
23
11 Jan 2022
24
19 Feb 2021
25
2 Aug 2021
26
2 Feb 2023
27
4 Mar 2021
28
5 Jan 2021
29
1 Aug 2021
30
18 Feb 2021
31
26 Nov 2022
32
2 Mar 2021
33
1 Feb 2023
34
31 Jul 2020
35
24 Nov 2022
36
2 Nov 2021
37
2 Feb 2023
38
2 Feb 2023
39
29 Jan 2023
40
1 Feb 2023
41
31 Jan 2023
42
28 Apr 2022
43
20 Jun 2022
44
22 Jul 2022
45
28 Feb 2021

46
23 Jul 2021
47
23 Jul 2021
48
18 Mar 2022
49
30 Jan 2023
50
31 Jan 2023
51
8 Dec 2021
52
24 Jun 2021
53
27 Feb 2022
54
2 Jan 2023
55
14 Jan 2022
56
15 May 2022
57
23 Jul 2021
58
15 Feb 2021
59
3 Nov 2021
60
7 Jul 2021
61
3 Jul 2021
62
18 Dec 2022
63
30 Jan 2022
64
11 May 2021
65
6 Jan 2023
66
21 Jul 2021
67
7 Jul 2022
68
15 Jun 2022
69
26 Nov 2022

70
26 Nov 2021
71
10 May 2022
72
9 Aug 2022
73
8 Jul 2022
74
3 Jul 2023
75
31 May 2022
76
3 Aug 2022
77
31 Jan 2023
78
17 Jan 2022
79
16 Mar 2023
80
3 Mar 2021
81
30 Sep 2022
82
1 Mar 2021
83
6 Jun 2021
84
28 May 2021
85
5 Mar 2021
86
31 May 2021
87
9 Mar 2022
88
10 Feb 2021
89
1 Mar 2021
90
5 Sep 2021
91
14 Jun 2021
92
30 Mar 2022
93
17 Jul 2021

94
14 Apr 2022
95
31 Jan 2023
96
12 May 2022
97
19 Feb 2021
98
29 Nov 2022
99
7 Sep 2021
100
13 Mar 2022
101
7 Jul 2021
102
8 Sep 2021
103
16 Apr 2021
104
22 Nov 2020
105
15 Oct 2021
106
20 Apr 2022
107
10 Jul 2021
108
10 May 2021
109
6 Jan 2023
110
22 Jul 2021
111
16 Sep 2021
112
4 Jul 2022
113
26 Jul 2022
114
6 Jul 2021
115
3 Sep 2021
116
29 Jan 2023
117
22 Feb 2021

118
28 Feb 2021
119
25 Nov 2020
120
1 Jul 2021
121
12 Jul 2022
122
20 Jan 2022
123
28 Oct 2020
124
5 Mar 2021
125
5 Feb 2022
126
28 Jan 2023
127
17 Feb 2021
128
27 Mar 2022
129
17 Nov 2022
130
7 Jan 2023
131
27 Apr 2022
132
5 Jan 2022
133
11 Nov 2022
134
29 Jan 2021
135
6 Jun 2022
136
6 Oct 2021
137
26 Aug 2021
138
7 Oct 2022
139
28 Jan 2023
140
29 Jan 2023
141
26 Apr 2022

142
12 Jul 2021
143
2 Feb 2023
144
3 Aug 2021
145
2 Feb 2023
146
2 Feb 2023
147
24 May 2021
148
1 Mar 2021
149
26 May 2021
150
1 Jul 2021
151
30 Mar 2021
152
7 Jan 2021
153
24 May 2021
154
7 Nov 2022
155
3 Feb 2023
156
18 Nov 2020
157
4 Mar 2021
158
6 Jan 2023
159
3 Jan 2022
160
23 Aug 2021
161
2 Jul 2021
162
11 Feb 2021
163
24 Nov 2021
164
1 Feb 2023
165
19 May 2022

166
 29 Jul 2022
 167
 16 Apr 2022
 168
 7 Sep 2020
 169
 30 Mar 2021
 170
 28 Aug 2022
 171
 10 Mar 2022
 172
 15 Oct 2022

173 .mw-parser-output .reflist-columns-2{column-width:30em}.mw-parser-output .reflist-columns-3{column-width:25em}body.skin-vector-2022 .mw-parser-output .reflist-columns-2{column-width:27em}body.skin-vector-2022 .mw-parser-output .reflist-columns-3{column-width:22.5em}.mw-parser-output .references[data-mw-group=upper-alpha]{list-style-type:upper-alpha}.mw-parser-output .references[data-mw-group=upper-roman]{list-style-type:upper-roman}.mw-parser-output .references[data-mw-group=lower-alpha]{list-style-type:lower-alpha}.mw-parser-output .references[data-mw-group=lower-greek]{list-style-type:lower-greek}.mw-parser-output .references[data-mw-group=lower-roman]{list-style-type:lower-roman}.mw-parser-output div.reflist-liststyle-upper-alpha .references{list-style-type:upper-alpha}.mw-parser-output div.reflist-liststyle-upper-roman .references{list-style-type:upper-roman}.mw-parser-output div.reflist-liststyle-lower-alpha .references{list-style-type:lower-alpha}.mw-parser-output div.reflist-liststyle-lower-greek .references{list-style-type:lower-greek}.mw-parser-output div.reflist-liststyle-lower-roman .references{list-style-type:lower-roman}\n^n^ Local time.\n\n^n^ For some countries it is unclear whether they report samples or cases. One person tested twice is recorded as one case and two samples.\n\n^n^ Excluding Taiwan.\n\n^n^ Excluding Northern Cyprus.\n\n^n^ Excluding Greenland and the Faroe Islands.\n\n^n^ Excluding Overseas France.\n\n^n^ Testing data from 4 May to 12 May is missing because of the transition to the new reporting system SI-DEP.\n\n^n^ Excluding Abkhazia and South Ossetia.\n\n^n^ Data for residents only.\n\n^n^ Excluding Transnistria.\n\n^n^ Northern Cyprus is not recognized as a sovereign state by any country except Turkey.\n\n^n^ Includes data for Liechtenstein.\n\n^n^ Not a United Nations member.

	tested	confirmed	confirmed	tested	ratio	tested	population	ratio
1	154767	NA		49621			32.100	
2	428654	NA		96838			22.600	
3	230553	NA		58574			25.400	
4	300307	NA		37958			12.600	
5	399228	NA		20981			5.300	
6	15268	NA		832			5.400	
7	35716069	NA		9060495			25.400	
8	3099602	NA		422963			13.600	
9	78548492	NA		10112229			12.900	

10	205817752	NA	5789991	2.800
11	6838458	NA	792638	11.600
12	259366	NA	37483	14.500
13	10578766	NA	696614	6.600
14	7417714	NA	1151644	15.500
15	770100	NA	103014	13.400
16	13217569	NA	982809	7.400
17	36548544	NA	4691499	12.800
18	572900	NA	60694	10.600
19	595112	NA	7884	1.300
20	1736168	NA	12702	0.730
21	4358669	NA	910228	20.900
22	1872934	NA	399887	21.400
23	2026898	NA	232432	11.500
24	23561497	NA	10081676	42.800
25	153804	NA	338	0.220
26	10993239	NA	1295524	11.800
27	158777	NA	12123	7.600
28	90019	NA	884	0.980
29	1812706	NA	77914	4.300
30	942685	NA	32681	3.500
31	66343123	NA	4423053	6.700
32	99027	NA	4020	4.100
33	48154268	NA	5123007	10.600
34	160000000	NA	87655	0.055
35	36875818	NA	6314769	17.100
36	2575363	NA	561054	21.800
37	5481285	NA	1267798	23.100
38	14301394	NA	1112470	7.800
39	27820163	NA	644160	2.300
40	22544928	NA	4590529	20.400
41	67682707	NA	3399947	5.000
42	305941	NA	15631	5.100
43	209803	NA	14821	7.100
44	3574665	NA	626030	17.500
45	124838	NA	25961	20.800
46	1627189	NA	480720	29.500
47	3137519	NA	283947	9.100
48	1847861	NA	161052	8.700
49	403773	NA	17113	4.200
50	3637908	NA	613954	16.900
51	415110	NA	49253	11.900
52	2981185	NA	278446	9.300
53	774000	NA	34237	4.400
54	667953	NA	68848	10.300
55	9042453	NA	371135	4.100
56	272417258	NA	29183646	10.700
57	958807	NA	25325	2.600

58	43217	NA	4469	10.300
59	4888787	NA	732965	15.000
60	65247345	NA	3733519	5.700
61	1305749	NA	96708	7.400
62	101576831	NA	5548487	5.500
63	164573	NA	10662	6.500
64	28684	NA	161	0.560
65	6800560	NA	1230098	18.100
66	494898	NA	24878	5.000
67	145231	NA	8400	5.800
68	648569	NA	66129	10.200
69	223475	NA	33874	15.200
70	1133782	NA	377859	33.300
71	11394556	NA	1909948	16.800
72	1988652	NA	203162	10.200
73	866177937	NA	43585554	5.000
74	76062770	NA	6812127	9.000
75	52269202	NA	7232268	13.800
76	19090652	NA	2448484	12.800
77	12990476	NA	1700817	13.100
78	41373364	NA	1792137	4.300
79	269127054	NA	25651205	9.500
80	429177	NA	33285	7.800
81	1184973	NA	151931	12.800
82	8487288	NA	432773	5.100
83	7407053	NA	739847	10.000
84	11575012	NA	385144	3.300
85	1322806	NA	107729	8.100
86	611357	NA	107410	17.600
87	7754247	NA	624573	8.100
88	695415	NA	85253	12.300
89	114030	NA	45	0.039
90	3630095	NA	144518	4.000
91	4599186	NA	542649	11.800
92	431221	NA	32910	7.600
93	128246	NA	5396	4.200
94	2578215	NA	501862	19.500
95	9046584	NA	1170108	12.900
96	4248188	NA	244182	5.700
97	119608	NA	19831	16.600
98	624784	NA	88086	14.100
99	23705425	NA	1880734	7.900
100	2216560	NA	174658	7.900
101	322504	NA	14449	4.500
102	1211456	NA	36606	3.000
103	268093	NA	18103	6.800
104	289552	NA	494	0.170
105	10503678	NA	3749860	35.700

106	3213594	NA	516864	16.100
107	3354200	NA	136053	4.100
108	394388	NA	98449	25.000
109	14217563	NA	1272299	8.900
110	688570	NA	105866	15.400
111	4047680	NA	440741	10.900
112	1062663	NA	166229	15.600
113	5804358	NA	984475	17.000
114	14526293	NA	1692834	11.700
115	41962	NA	136	0.320
116	7757935	NA	2136662	27.500
117	79321	NA	4740	6.000
118	1544008	NA	155657	10.100
119	16914	NA	0	0.000
120	881870	NA	155689	17.700
121	7096998	NA	103034	1.500
122	9811888	NA	554778	5.700
123	509959	NA	114434	22.400
124	9173593	NA	588728	6.400
125	3078533	NA	574105	18.600
126	7475016	NA	1029701	13.800
127	47490	NA	961	2.000
128	2609819	NA	647950	24.800
129	36073768	NA	4177786	11.600
130	34402980	NA	4073980	11.800
131	36064311	NA	5993861	16.600
132	27515490	NA	1499976	5.500
133	4061988	NA	473440	11.700
134	5405393	NA	724250	13.400
135	295542733	NA	18358459	6.200
136	2885812	NA	98209	3.400
137	30231	NA	995	3.300
138	212132	NA	29550	13.900
139	113504	NA	9585	8.400
140	192613	NA	23427	12.200
141	41849069	NA	753632	1.800
142	624502	NA	46509	7.400
143	12185475	NA	2473599	20.300
144	16206203	NA	65315	0.400
145	7391882	NA	1861034	25.200
146	2826117	NA	1322282	46.800
147	11378282	NA	1637848	14.400
148	6592010	NA	90029	1.400
149	164472	NA	10688	6.500
150	54128524	NA	3821305	7.100
151	2384745	NA	93128	3.900
152	158804	NA	23316	14.700
153	9996795	NA	1074751	10.800

154	23283909	NA	4276836	18.400
155	30275725	NA	8622129	28.480
156	3880	NA	509	13.100
157	1579597	NA	26162	1.700
158	807269	NA	39358	4.900
159	512730	NA	92997	18.100
160	2893625	NA	703732	24.300
161	61236294	NA	5435831	8.900
162	852444	NA	39979	4.700
163	15648456	NA	3367461	21.500
164	198685717	NA	1049537	0.530
165	522526476	NA	22232377	4.300
166	929349291	NA	90749469	9.800
167	6089116	NA	895592	14.700
168	2630000	NA	43975	1.700
169	3179074	NA	159149	5.000
170	45772571	NA	11403302	24.900
171	3301860	NA	314850	9.500
172	2529087	NA	257893	10.200
173	NA	NA	NA	NA

confirmed.population.ratio

1	0.4000
2	15.0000
3	0.5300
4	387.0000
5	1.3000
6	15.9000
7	78.3000
8	105.0000
9	313.0000
10	2312.0000
11	69.1000
12	67.3000
13	674.0000
14	4.5000
15	268.0000
16	139.0000
17	317.0000
18	140.0000
19	5.1000
20	234.0000
21	38.1000
22	54.7000
23	89.9000
24	11.2000
25	33.5000
26	158.0000
27	0.7600

28	0.7600
29	11.2000
30	3.6000
31	175.0000
32	0.7200
33	252.0000
34	11.1000
35	76.4000
36	51.5000
37	134.0000
38	126.0000
39	3223.0000
40	211.0000
41	1162.0000
42	33.2000
43	293.0000
44	32.9000
45	0.1400
46	9.5000
47	3.1000
48	28.5000
49	30.8000
50	274.0000
51	36.5000
52	2.6000
53	1493.0000
54	74.5000
55	163.0000
56	417.0000
57	3.1000
58	2.0000
59	132.0000
60	77.8000
61	4.2000
62	943.0000
63	293.0000
64	25.7000
65	39.4000
66	3.8000
67	7.7000
68	82.5000
69	2.0000
70	11.8000
71	118.0000
72	546.0000
73	63.0000
74	28.2000
75	62.8000

76	47.5000
77	264.0000
78	451.0000
79	446.0000
80	1.6000
81	43.5000
82	6.7000
83	69.5000
84	62.1000
85	2.8000
86	33.8000
87	181.0000
88	10.7000
89	1.6000
90	189.0000
91	67.4000
92	21.5000
93	2.5000
94	37.6000
95	324.0000
96	679.0000
97	0.4600
98	3.3000
99	72.3000
100	398.0000
101	1.6000
102	245.0000
103	6.1000
104	22.9000
105	8.2000
106	122.0000
107	100.0000
108	62.5000
109	38.5000
110	2.2000
111	7.4000
112	38.7000
113	20.7000
114	83.4000
115	15.7000
116	156.0000
117	0.3500
118	0.7500
119	0.0660
120	42.5000
121	2177.0000
122	183.0000
123	11.0000

124	4.2000
125	60.9000
126	179.0000
127	0.5300
128	36.6000
129	109.9000
130	34.1000
131	94.0000
132	268.0000
133	141.0000
134	27.9000
135	201.0000
136	22.3000
137	57.6000
138	116.6000
139	103.0000
140	563.0000
141	120.0000
142	3.9000
143	175.0000
144	284.0000
145	135.0000
146	135.0000
147	19.2000
148	12.7000
149	1.3000
150	116.0000
151	10.9000
152	0.3600
153	96.8000
154	270.0000
155	128.3000
156	0.0065
157	2.3000
158	9.4000
159	37.6000
160	24.5000
161	73.6000
162	1.9000
163	37.2000
164	2070.0000
165	774.0000
166	281.0000
167	175.0000
168	7.7000
169	11.0000
170	46.4000
171	19.0000

```
172          17.0000
173          NA
```

After pre-processing, you can see the columns and columns names are simplified, and columns types are converted into correct types.

The data frame has following columns:

- **country** - The name of the country
- **date** - Reported date
- **tested** - Total tested cases by the reported date
- **confirmed** - Total confirmed cases by the reported date
- **confirmed.tested.ratio** - The ratio of confirmed cases to the tested cases
- **tested.population.ratio** - The ratio of tested cases to the population of the country
- **confirmed.population.ratio** - The ratio of confirmed cases to the population of the country

OK, we can call `write.csv()` function to save the csv file into a file.

```
[11]: # Export the data frame to a csv file
```

Note for IBM Watson Studio, there is no traditional “hard disk” associated with a R workspace.

Even if you call `write.csv()` method to save the data frame as a csv file, it won’t be shown in IBM Cloud Object Storage asset UI automatically.

However, you may still check if the `covid.csv` exists using following code snippet:

```
[12]: # Get working directory
wd <- getwd()
# Get exported
file_path <- paste(wd, sep="", "/covid.csv")
# File path
print(file_path)
file.exists(file_path)
```

```
[1] "/resources/RP0101EN/labs/M5/covid.csv"
```

TRUE

Optional Step: If you have difficulties finishing above webscraping tasks, you may still continue with next tasks by downloading a provided csv file from here:

```
[13]: ## Download a sample csv file
# covid_csv_file <- download.file("https://cf-courses-data.s3.us.
  ↪cloud-object-storage.appdomain.cloud/
  ↪IBMDeveloperSkillsNetwork-RP0101EN-Coursera/v2/dataset/covid.csv",
  ↪destfile="covid.csv")
# covid_data_frame_csv <- read.csv("covid.csv", header=TRUE, sep=",")
```

0.5 TASK 4: Get a subset of the extracted data frame

The goal of task 4 is to get the 5th to 10th rows from the data frame with only `country` and `confirmed` columns selected

```
[14]: # Read covid_data_frame_csv from the csv file
# Export dataframe first
write.csv(clean_covid_df, file = file_path, row.names = FALSE)
# Read CSV file
covid_data_frame_csv <- read.csv(file_path, header = TRUE)
# Print first rows
head(covid_data_frame_csv)

# Get the 5th to 10th rows, with two "country" "confirmed" columns
covid_data_frame_csv[5:10, c("country", "confirmed")]
```

A data.frame: 6 × 7

	country <chr>	date <chr>	tested <dbl>	confirmed <lgl>	confirmed.tested.ratio <int>	test <d
1	Afghanistan	17 Dec 2020	154767	NA	49621	32.
2	Albania	18 Feb 2021	428654	NA	96838	22.
3	Algeria	2 Nov 2020	230553	NA	58574	25.
4	Andorra	23 Feb 2022	300307	NA	37958	12.
5	Angola	2 Feb 2021	399228	NA	20981	5.3
6	Antigua and Barbuda	6 Mar 2021	15268	NA	832	5.4

A data.frame: 6 × 2

	country <chr>	confirmed <lgl>
5	Angola	NA
6	Antigua and Barbuda	NA
7	Argentina	NA
8	Armenia	NA
9	Australia	NA
10	Austria	NA

0.6 TASK 5: Calculate worldwide COVID testing positive ratio

The goal of task 5 is to get the total confirmed and tested cases worldwide, and try to figure the overall positive ratio using `confirmed cases / tested cases`

```
[15]: # Get the total confirmed cases worldwide
total_confirmed <- sum(
  covid_data_frame_csv$confirmed,
  na.rm = TRUE
)
print(total_confirmed)

# Get the total tested cases worldwide
total_tested <- sum(
  covid_data_frame_csv$tested,
  na.rm = TRUE
)
print(total_tested)
```



```
# Get the positive ratio (confirmed / tested)
positive_ratio <- total_confirmed / total_tested
print(positive_ratio)
```

```
[1] 0
[1] 5396881644
[1] 0
```

0.7 TASK 6: Get a country list which reported their testing data

The goal of task 6 is to get a catalog or sorted list of countries who have reported their COVID-19 testing data

```
[16]: # Get the `country` column
country_column <- covid_data_frame_csv$country
print(country_column)

# Check its class (should be character)
class(country_column)

# Sort the countries AtoZ
countries_atoz <- sort(country_column)
print(countries_atoz)

# Sort the countries ZtoA
countries_ztoa <- sort(country_column, decreasing = TRUE)
print(countries_ztoa)

# Print the sorted ZtoA list
print(countries_ztoa)
```

```
[1] "Afghanistan"
[2] "Albania"
[3] "Algeria"
[4] "Andorra"
[5] "Angola"
[6] "Antigua and Barbuda"
[7] "Argentina"
[8] "Armenia"
[9] "Australia"
[10] "Austria"
[11] "Azerbaijan"
[12] "Bahamas"
[13] "Bahrain"
[14] "Bangladesh"
[15] "Barbados"
[16] "Belarus"
[17] "Belgium"
[18] "Belize"
```

[19] "Benin"
[20] "Bhutan"
[21] "Bolivia"
[22] "Bosnia and Herzegovina"
[23] "Botswana"
[24] "Brazil"
[25] "Brunei"
[26] "Bulgaria"
[27] "Burkina Faso"
[28] "Burundi"
[29] "Cambodia"
[30] "Cameroon"
[31] "Canada"
[32] "Chad"
[33] "Chile"
[34] "China[c] "
[35] "Colombia"
[36] "Costa Rica"
[37] "Croatia"
[38] "Cuba"
[39] "Cyprus[d] "
[40] "Czechia"
[41] "Denmark[e] "
[42] "Djibouti"
[43] "Dominica"
[44] "Dominican Republic"
[45] "DR Congo"
[46] "Ecuador"
[47] "Egypt"
[48] "El Salvador"
[49] "Equatorial Guinea"
[50] "Estonia"
[51] "Eswatini"
[52] "Ethiopia"
[53] "Faroe Islands"
[54] "Fiji"
[55] "Finland"
[56] "France[f] [g] "
[57] "Gabon"
[58] "Gambia"
[59] "Georgia[h] "
[60] "Germany"
[61] "Ghana"
[62] "Greece"
[63] "Greenland"
[64] "Grenada"
[65] "Guatemala"
[66] "Guinea"

[67] "Guinea-Bissau"
[68] "Guyana"
[69] "Haiti"
[70] "Honduras"
[71] "Hungary"
[72] "Iceland"
[73] "India"
[74] "Indonesia"
[75] "Iran"
[76] "Iraq"
[77] "Ireland"
[78] "Israel"
[79] "Italy"
[80] "Ivory Coast"
[81] "Jamaica"
[82] "Japan"
[83] "Jordan"
[84] "Kazakhstan"
[85] "Kenya"
[86] "Kosovo"
[87] "Kuwait"
[88] "Kyrgyzstan"
[89] "Laos"
[90] "Latvia"
[91] "Lebanon"
[92] "Lesotho"
[93] "Liberia"
[94] "Libya"
[95] "Lithuania"
[96] "Luxembourg[i]"
[97] "Madagascar"
[98] "Malawi"
[99] "Malaysia"
[100] "Maldives"
[101] "Mali"
[102] "Malta"
[103] "Mauritania"
[104] "Mauritius"
[105] "Mexico"
[106] "Moldova[j]"
[107] "Mongolia"
[108] "Montenegro"
[109] "Morocco"
[110] "Mozambique"
[111] "Myanmar"
[112] "Namibia"
[113] "Nepal"
[114] "Netherlands"

[115] "New Caledonia"
[116] "New Zealand"
[117] "Niger"
[118] "Nigeria"
[119] "North Korea"
[120] "North Macedonia"
[121] "Northern Cyprus[k] "
[122] "Norway"
[123] "Oman"
[124] "Pakistan"
[125] "Palestine"
[126] "Panama"
[127] "Papua New Guinea"
[128] "Paraguay"
[129] "Peru"
[130] "Philippines"
[131] "Poland"
[132] "Portugal"
[133] "Qatar"
[134] "Romania"
[135] "Russia"
[136] "Rwanda"
[137] "Saint Kitts and Nevis"
[138] "Saint Lucia"
[139] "Saint Vincent"
[140] "San Marino"
[141] "Saudi Arabia"
[142] "Senegal"
[143] "Serbia"
[144] "Singapore"
[145] "Slovakia"
[146] "Slovenia"
[147] "South Africa"
[148] "South Korea"
[149] "South Sudan"
[150] "Spain"
[151] "Sri Lanka"
[152] "Sudan"
[153] "Sweden"
[154] "Switzerland[l] "
[155] "Taiwan[m] "
[156] "Tanzania"
[157] "Thailand"
[158] "Togo"
[159] "Trinidad and Tobago"
[160] "Tunisia"
[161] "Turkey"
[162] "Uganda"

[163] "Ukraine"
 [164] "United Arab Emirates"
 [165] "United Kingdom"
 [166] "United States"
 [167] "Uruguay"
 [168] "Uzbekistan"
 [169] "Venezuela"
 [170] "Vietnam"
 [171] "Zambia"
 [172] "Zimbabwe"
 [173] ".mw-parser-output .reflist-columns-2{column-width:30em}.mw-parser-output .reflist-columns-3{column-width:25em}body.skin-vector-2022 .mw-parser-output .reflist-columns-2{column-width:27em}body.skin-vector-2022 .mw-parser-output .reflist-columns-3{column-width:22.5em}.mw-parser-output .references[data-mw-group=upper-alpha]{list-style-type:upper-alpha}.mw-parser-output .references[data-mw-group=upper-roman]{list-style-type:upper-roman}.mw-parser-output .references[data-mw-group=lower-alpha]{list-style-type:lower-alpha}.mw-parser-output .references[data-mw-group=lower-greek]{list-style-type:lower-greek}.mw-parser-output .references[data-mw-group=lower-roman]{list-style-type:lower-roman}.mw-parser-output div.reflist-liststyle-upper-alpha .references{list-style-type:upper-alpha}.mw-parser-output div.reflist-liststyle-upper-roman .references{list-style-type:upper-roman}.mw-parser-output div.reflist-liststyle-lower-alpha .references{list-style-type:lower-alpha}.mw-parser-output div.reflist-liststyle-lower-greek .references{list-style-type:lower-greek}.mw-parser-output div.reflist-liststyle-lower-roman .references{list-style-type:lower-roman}\n^ Local time.\n\n^ For some countries it is unclear whether they report samples or cases. One person tested twice is recorded as one case and two samples.\n\n^ Excluding Taiwan.\n\n^ Excluding Northern Cyprus.\n\n^ Excluding Greenland and the Faroe Islands.\n\n^ Excluding Overseas France.\n\n^ Testing data from 4 May to 12 May is missing because of the transition to the new reporting system SI-DEP.\n\n^ Excluding Abkhazia and South Ossetia.\n\n^ Data for residents only.\n\n^ Excluding Transnistria.\n\n^ Northern Cyprus is not recognized as a sovereign state by any country except Turkey.\n\n^ Includes data for Liechtenstein.\n\n^ Not a United Nations member."

'character'

[1] ".mw-parser-output .reflist-columns-2{column-width:30em}.mw-parser-output .reflist-columns-3{column-width:25em}body.skin-vector-2022 .mw-parser-output .reflist-columns-2{column-width:27em}body.skin-vector-2022 .mw-parser-output .reflist-columns-3{column-width:22.5em}.mw-parser-output .references[data-mw-group=upper-alpha]{list-style-type:upper-alpha}.mw-parser-output .references[data-mw-group=upper-roman]{list-style-type:upper-roman}.mw-parser-output .references[data-mw-group=lower-alpha]{list-style-type:lower-alpha}.mw-parser-output .references[data-mw-group=lower-greek]{list-style-type:lower-greek}.mw-parser-output .references[data-mw-group=lower-roman]{list-style-type:lower-roman}.mw-parser-output div.reflist-liststyle-upper-alpha .references{list-style-type:upper-alpha}.mw-parser-output div.reflist-liststyle-upper-roman .references{list-style-type:upper-roman}.mw-parser-output

div.reflist-liststyle-lower-alpha .references{list-style-type:lower-alpha}.mw-parser-output div.reflist-liststyle-lower-greek .references{list-style-type:lower-greek}.mw-parser-output div.reflist-liststyle-lower-roman .references{list-style-type:lower-roman}\n^ Local time.\n\n^ For some countries it is unclear whether they report samples or cases. One person tested twice is recorded as one case and two samples.\n\n^ Excluding Taiwan.\n\n^ Excluding Northern Cyprus.\n\n^ Excluding Greenland and the Faroe Islands.\n\n^ Excluding Overseas France.\n\n^ Testing data from 4 May to 12 May is missing because of the transition to the new reporting system SI-DEP.\n\n^ Excluding Abkhazia and South Ossetia.\n\n^ Data for residents only.\n\n^ Excluding Transnistria.\n\n^ Northern Cyprus is not recognized as a sovereign state by any country except Turkey.\n\n^ Includes data for Liechtenstein.\n\n^ Not a United Nations member."

- [2] "Afghanistan"
- [3] "Albania"
- [4] "Algeria"
- [5] "Andorra"
- [6] "Angola"
- [7] "Antigua and Barbuda"
- [8] "Argentina"
- [9] "Armenia"
- [10] "Australia"
- [11] "Austria"
- [12] "Azerbaijan"
- [13] "Bahamas"
- [14] "Bahrain"
- [15] "Bangladesh"
- [16] "Barbados"
- [17] "Belarus"
- [18] "Belgium"
- [19] "Belize"
- [20] "Benin"
- [21] "Bhutan"
- [22] "Bolivia"
- [23] "Bosnia and Herzegovina"
- [24] "Botswana"
- [25] "Brazil"
- [26] "Brunei"
- [27] "Bulgaria"
- [28] "Burkina Faso"
- [29] "Burundi"
- [30] "Cambodia"
- [31] "Cameroon"
- [32] "Canada"
- [33] "Chad"
- [34] "Chile"
- [35] "China[c]"
- [36] "Colombia"
- [37] "Costa Rica"

[38] "Croatia"
[39] "Cuba"
[40] "Cyprus[d] "
[41] "Czechia"
[42] "Denmark[e] "
[43] "Djibouti"
[44] "Dominica"
[45] "Dominican Republic"
[46] "DR Congo"
[47] "Ecuador"
[48] "Egypt"
[49] "El Salvador"
[50] "Equatorial Guinea"
[51] "Estonia"
[52] "Eswatini"
[53] "Ethiopia"
[54] "Faroe Islands"
[55] "Fiji"
[56] "Finland"
[57] "France[f] [g] "
[58] "Gabon"
[59] "Gambia"
[60] "Georgia[h] "
[61] "Germany"
[62] "Ghana"
[63] "Greece"
[64] "Greenland"
[65] "Grenada"
[66] "Guatemala"
[67] "Guinea"
[68] "Guinea-Bissau"
[69] "Guyana"
[70] "Haiti"
[71] "Honduras"
[72] "Hungary"
[73] "Iceland"
[74] "India"
[75] "Indonesia"
[76] "Iran"
[77] "Iraq"
[78] "Ireland"
[79] "Israel"
[80] "Italy"
[81] "Ivory Coast"
[82] "Jamaica"
[83] "Japan"
[84] "Jordan"
[85] "Kazakhstan"

[86] "Kenya"
[87] "Kosovo"
[88] "Kuwait"
[89] "Kyrgyzstan"
[90] "Laos"
[91] "Latvia"
[92] "Lebanon"
[93] "Lesotho"
[94] "Liberia"
[95] "Libya"
[96] "Lithuania"
[97] "Luxembourg[i] "
[98] "Madagascar"
[99] "Malawi"
[100] "Malaysia"
[101] "Maldives"
[102] "Mali"
[103] "Malta"
[104] "Mauritania"
[105] "Mauritius"
[106] "Mexico"
[107] "Moldova[j] "
[108] "Mongolia"
[109] "Montenegro"
[110] "Morocco"
[111] "Mozambique"
[112] "Myanmar"
[113] "Namibia"
[114] "Nepal"
[115] "Netherlands"
[116] "New Caledonia"
[117] "New Zealand"
[118] "Niger"
[119] "Nigeria"
[120] "North Korea"
[121] "North Macedonia"
[122] "Northern Cyprus[k] "
[123] "Norway"
[124] "Oman"
[125] "Pakistan"
[126] "Palestine"
[127] "Panama"
[128] "Papua New Guinea"
[129] "Paraguay"
[130] "Peru"
[131] "Philippines"
[132] "Poland"
[133] "Portugal"

- [134] "Qatar"
- [135] "Romania"
- [136] "Russia"
- [137] "Rwanda"
- [138] "Saint Kitts and Nevis"
- [139] "Saint Lucia"
- [140] "Saint Vincent"
- [141] "San Marino"
- [142] "Saudi Arabia"
- [143] "Senegal"
- [144] "Serbia"
- [145] "Singapore"
- [146] "Slovakia"
- [147] "Slovenia"
- [148] "South Africa"
- [149] "South Korea"
- [150] "South Sudan"
- [151] "Spain"
- [152] "Sri Lanka"
- [153] "Sudan"
- [154] "Sweden"
- [155] "Switzerland[1]"
- [156] "Taiwan[m]"
- [157] "Tanzania"
- [158] "Thailand"
- [159] "Togo"
- [160] "Trinidad and Tobago"
- [161] "Tunisia"
- [162] "Turkey"
- [163] "Uganda"
- [164] "Ukraine"
- [165] "United Arab Emirates"
- [166] "United Kingdom"
- [167] "United States"
- [168] "Uruguay"
- [169] "Uzbekistan"
- [170] "Venezuela"
- [171] "Vietnam"
- [172] "Zambia"
- [173] "Zimbabwe"
- [1] "Zimbabwe"
- [2] "Zambia"
- [3] "Vietnam"
- [4] "Venezuela"
- [5] "Uzbekistan"
- [6] "Uruguay"
- [7] "United States"
- [8] "United Kingdom"

[9] "United Arab Emirates"
[10] "Ukraine"
[11] "Uganda"
[12] "Turkey"
[13] "Tunisia"
[14] "Trinidad and Tobago"
[15] "Togo"
[16] "Thailand"
[17] "Tanzania"
[18] "Taiwan[m] "
[19] "Switzerland[l] "
[20] "Sweden"
[21] "Sudan"
[22] "Sri Lanka"
[23] "Spain"
[24] "South Sudan"
[25] "South Korea"
[26] "South Africa"
[27] "Slovenia"
[28] "Slovakia"
[29] "Singapore"
[30] "Serbia"
[31] "Senegal"
[32] "Saudi Arabia"
[33] "San Marino"
[34] "Saint Vincent"
[35] "Saint Lucia"
[36] "Saint Kitts and Nevis"
[37] "Rwanda"
[38] "Russia"
[39] "Romania"
[40] "Qatar"
[41] "Portugal"
[42] "Poland"
[43] "Philippines"
[44] "Peru"
[45] "Paraguay"
[46] "Papua New Guinea"
[47] "Panama"
[48] "Palestine"
[49] "Pakistan"
[50] "Oman"
[51] "Norway"
[52] "Northern Cyprus[k] "
[53] "North Macedonia"
[54] "North Korea"
[55] "Nigeria"
[56] "Niger"

[57] "New Zealand"
[58] "New Caledonia"
[59] "Netherlands"
[60] "Nepal"
[61] "Namibia"
[62] "Myanmar"
[63] "Mozambique"
[64] "Morocco"
[65] "Montenegro"
[66] "Mongolia"
[67] "Moldova[j]"
[68] "Mexico"
[69] "Mauritius"
[70] "Mauritania"
[71] "Malta"
[72] "Mali"
[73] "Maldives"
[74] "Malaysia"
[75] "Malawi"
[76] "Madagascar"
[77] "Luxembourg[i]"
[78] "Lithuania"
[79] "Libya"
[80] "Liberia"
[81] "Lesotho"
[82] "Lebanon"
[83] "Latvia"
[84] "Laos"
[85] "Kyrgyzstan"
[86] "Kuwait"
[87] "Kosovo"
[88] "Kenya"
[89] "Kazakhstan"
[90] "Jordan"
[91] "Japan"
[92] "Jamaica"
[93] "Ivory Coast"
[94] "Italy"
[95] "Israel"
[96] "Ireland"
[97] "Iraq"
[98] "Iran"
[99] "Indonesia"
[100] "India"
[101] "Iceland"
[102] "Hungary"
[103] "Honduras"
[104] "Haiti"

[105] "Guyana"
[106] "Guinea-Bissau"
[107] "Guinea"
[108] "Guatemala"
[109] "Grenada"
[110] "Greenland"
[111] "Greece"
[112] "Ghana"
[113] "Germany"
[114] "Georgia[h] "
[115] "Gambia"
[116] "Gabon"
[117] "France[f] [g] "
[118] "Finland"
[119] "Fiji"
[120] "Faroe Islands"
[121] "Ethiopia"
[122] "Eswatini"
[123] "Estonia"
[124] "Equatorial Guinea"
[125] "El Salvador"
[126] "Egypt"
[127] "Ecuador"
[128] "DR Congo"
[129] "Dominican Republic"
[130] "Dominica"
[131] "Djibouti"
[132] "Denmark[e] "
[133] "Czechia"
[134] "Cyprus[d] "
[135] "Cuba"
[136] "Croatia"
[137] "Costa Rica"
[138] "Colombia"
[139] "China[c] "
[140] "Chile"
[141] "Chad"
[142] "Canada"
[143] "Cameroon"
[144] "Cambodia"
[145] "Burundi"
[146] "Burkina Faso"
[147] "Bulgaria"
[148] "Brunei"
[149] "Brazil"
[150] "Botswana"
[151] "Bosnia and Herzegovina"
[152] "Bolivia"

[153] "Bhutan"

[154] "Benin"

[155] "Belize"

[156] "Belgium"

[157] "Belarus"

[158] "Barbados"

[159] "Bangladesh"

[160] "Bahrain"

[161] "Bahamas"

[162] "Azerbaijan"

[163] "Austria"

[164] "Australia"

[165] "Armenia"

[166] "Argentina"

[167] "Antigua and Barbuda"

[168] "Angola"

[169] "Andorra"

[170] "Algeria"

[171] "Albania"

[172] "Afghanistan"

[173] ".mw-parser-output .reflist-columns-2{column-width:30em}.mw-parser-output .reflist-columns-3{column-width:25em}body.skin-vector-2022 .mw-parser-output .reflist-columns-2{column-width:27em}body.skin-vector-2022 .mw-parser-output .reflist-columns-3{column-width:22.5em}.mw-parser-output .references[data-mw-group=upper-alpha]{list-style-type:upper-alpha}.mw-parser-output .references[data-mw-group=upper-roman]{list-style-type:upper-roman}.mw-parser-output .references[data-mw-group=lower-alpha]{list-style-type:lower-alpha}.mw-parser-output .references[data-mw-group=lower-greek]{list-style-type:lower-greek}.mw-parser-output .references[data-mw-group=lower-roman]{list-style-type:lower-roman}.mw-parser-output div.reflist-liststyle-upper-alpha .references{list-style-type:upper-alpha}.mw-parser-output div.reflist-liststyle-upper-roman .references{list-style-type:upper-roman}.mw-parser-output div.reflist-liststyle-lower-alpha .references{list-style-type:lower-alpha}.mw-parser-output div.reflist-liststyle-lower-greek .references{list-style-type:lower-greek}.mw-parser-output div.reflist-liststyle-lower-roman .references{list-style-type:lower-roman}\n^ Local time.\n\n^ For some countries it is unclear whether they report samples or cases. One person tested twice is recorded as one case and two samples.\n\n^ Excluding Taiwan.\n\n^ Excluding Northern Cyprus.\n\n^ Excluding Greenland and the Faroe Islands.\n\n^ Excluding Overseas France.\n\n^ Testing data from 4 May to 12 May is missing because of the transition to the new reporting system SI-DEP.\n\n^ Excluding Abkhazia and South Ossetia.\n\n^ Data for residents only.\n\n^ Excluding Transnistria.\n\n^ Northern Cyprus is not recognized as a sovereign state by any country except Turkey.\n\n^ Includes data for Liechtenstein.\n\n^ Not a United Nations member."

[1] "Zimbabwe"

[2] "Zambia"

[3] "Vietnam"

[4] "Venezuela"

[5] "Uzbekistan"
[6] "Uruguay"
[7] "United States"
[8] "United Kingdom"
[9] "United Arab Emirates"
[10] "Ukraine"
[11] "Uganda"
[12] "Turkey"
[13] "Tunisia"
[14] "Trinidad and Tobago"
[15] "Togo"
[16] "Thailand"
[17] "Tanzania"
[18] "Taiwan[m] "
[19] "Switzerland[l] "
[20] "Sweden"
[21] "Sudan"
[22] "Sri Lanka"
[23] "Spain"
[24] "South Sudan"
[25] "South Korea"
[26] "South Africa"
[27] "Slovenia"
[28] "Slovakia"
[29] "Singapore"
[30] "Serbia"
[31] "Senegal"
[32] "Saudi Arabia"
[33] "San Marino"
[34] "Saint Vincent"
[35] "Saint Lucia"
[36] "Saint Kitts and Nevis"
[37] "Rwanda"
[38] "Russia"
[39] "Romania"
[40] "Qatar"
[41] "Portugal"
[42] "Poland"
[43] "Philippines"
[44] "Peru"
[45] "Paraguay"
[46] "Papua New Guinea"
[47] "Panama"
[48] "Palestine"
[49] "Pakistan"
[50] "Oman"
[51] "Norway"
[52] "Northern Cyprus[k] "

[53] "North Macedonia"
[54] "North Korea"
[55] "Nigeria"
[56] "Niger"
[57] "New Zealand"
[58] "New Caledonia"
[59] "Netherlands"
[60] "Nepal"
[61] "Namibia"
[62] "Myanmar"
[63] "Mozambique"
[64] "Morocco"
[65] "Montenegro"
[66] "Mongolia"
[67] "Moldova[j]"
[68] "Mexico"
[69] "Mauritius"
[70] "Mauritania"
[71] "Malta"
[72] "Mali"
[73] "Maldives"
[74] "Malaysia"
[75] "Malawi"
[76] "Madagascar"
[77] "Luxembourg[i]"
[78] "Lithuania"
[79] "Libya"
[80] "Liberia"
[81] "Lesotho"
[82] "Lebanon"
[83] "Latvia"
[84] "Laos"
[85] "Kyrgyzstan"
[86] "Kuwait"
[87] "Kosovo"
[88] "Kenya"
[89] "Kazakhstan"
[90] "Jordan"
[91] "Japan"
[92] "Jamaica"
[93] "Ivory Coast"
[94] "Italy"
[95] "Israel"
[96] "Ireland"
[97] "Iraq"
[98] "Iran"
[99] "Indonesia"
[100] "India"

[101] "Iceland"
[102] "Hungary"
[103] "Honduras"
[104] "Haiti"
[105] "Guyana"
[106] "Guinea-Bissau"
[107] "Guinea"
[108] "Guatemala"
[109] "Grenada"
[110] "Greenland"
[111] "Greece"
[112] "Ghana"
[113] "Germany"
[114] "Georgia[h] "
[115] "Gambia"
[116] "Gabon"
[117] "France[f] [g] "
[118] "Finland"
[119] "Fiji"
[120] "Faroe Islands"
[121] "Ethiopia"
[122] "Eswatini"
[123] "Estonia"
[124] "Equatorial Guinea"
[125] "El Salvador"
[126] "Egypt"
[127] "Ecuador"
[128] "DR Congo"
[129] "Dominican Republic"
[130] "Dominica"
[131] "Djibouti"
[132] "Denmark[e] "
[133] "Czechia"
[134] "Cyprus[d] "
[135] "Cuba"
[136] "Croatia"
[137] "Costa Rica"
[138] "Colombia"
[139] "China[c] "
[140] "Chile"
[141] "Chad"
[142] "Canada"
[143] "Cameroon"
[144] "Cambodia"
[145] "Burundi"
[146] "Burkina Faso"
[147] "Bulgaria"
[148] "Brunei"

[149] "Brazil"

[150] "Botswana"

[151] "Bosnia and Herzegovina"

[152] "Bolivia"

[153] "Bhutan"

[154] "Benin"

[155] "Belize"

[156] "Belgium"

[157] "Belarus"

[158] "Barbados"

[159] "Bangladesh"

[160] "Bahrain"

[161] "Bahamas"

[162] "Azerbaijan"

[163] "Austria"

[164] "Australia"

[165] "Armenia"

[166] "Argentina"

[167] "Antigua and Barbuda"

[168] "Angola"

[169] "Andorra"

[170] "Algeria"

[171] "Albania"

[172] "Afghanistan"

[173] ".mw-parser-output .reflist-columns-2{column-width:30em}.mw-parser-output .reflist-columns-3{column-width:25em}body.skin-vector-2022 .mw-parser-output .reflist-columns-2{column-width:27em}body.skin-vector-2022 .mw-parser-output .reflist-columns-3{column-width:22.5em}.mw-parser-output .references[data-mw-group=upper-alpha]{list-style-type:upper-alpha}.mw-parser-output .references[data-mw-group=upper-roman]{list-style-type:upper-roman}.mw-parser-output .references[data-mw-group=lower-alpha]{list-style-type:lower-alpha}.mw-parser-output .references[data-mw-group=lower-greek]{list-style-type:lower-greek}.mw-parser-output .references[data-mw-group=lower-roman]{list-style-type:lower-roman}.mw-parser-output div.reflist-liststyle-upper-alpha .references{list-style-type:upper-alpha}.mw-parser-output div.reflist-liststyle-upper-roman .references{list-style-type:upper-roman}.mw-parser-output div.reflist-liststyle-lower-alpha .references{list-style-type:lower-alpha}.mw-parser-output div.reflist-liststyle-lower-greek .references{list-style-type:lower-greek}.mw-parser-output div.reflist-liststyle-lower-roman .references{list-style-type:lower-roman}\n^ Local time.\n\n^ For some countries it is unclear whether they report samples or cases. One person tested twice is recorded as one case and two samples.\n\n^ Excluding Taiwan.\n\n^ Excluding Northern Cyprus.\n\n^ Excluding Greenland and the Faroe Islands.\n\n^ Excluding Overseas France.\n\n^ Testing data from 4 May to 12 May is missing because of the transition to the new reporting system SI-DEP.\n\n^ Excluding Abkhazia and South Ossetia.\n\n^ Data for residents only.\n\n^ Excluding Transnistria.\n\n^ Northern Cyprus is not recognized as a sovereign state by any country except Turkey.\n\n^ Includes data for Liechtenstein.\n\n^ Not a United Nations member."

0.8 TASK 7: Identify countries names with a specific pattern

The goal of task 7 is using a regular expression to find any countries start with United

```
[17]: # Use a regular expression `United.+` to find matches
```

```
matched_countries <- grep(
  "United.+",
  covid_data_frame_csv$country,
  value = TRUE
)
print(matched_countries)

# Print the matched country names
matched_countries <- grep(
  "United.+",
  covid_data_frame_csv$country,
  value = TRUE
)

print(matched_countries)
```

```
[1] "United Arab Emirates"
[2] "United Kingdom"
[3] "United States"
[4] ".mw-parser-output .reflist-columns-2{column-width:30em}.mw-parser-output
.reflist-columns-3{column-width:25em}body.skin-vector-2022 .mw-parser-output
.reflist-columns-2{column-width:27em}body.skin-vector-2022 .mw-parser-output
.reflist-columns-3{column-width:22.5em}.mw-parser-output .references[data-mw-
group=upper-alpha]{list-style-type:upper-alpha}.mw-parser-output
.references[data-mw-group=upper-roman]{list-style-type:upper-roman}.mw-parser-
output .references[data-mw-group=lower-alpha]{list-style-type:lower-alpha}.mw-
parser-output .references[data-mw-group=lower-greek]{list-style-type:lower-
greek}.mw-parser-output .references[data-mw-group=lower-roman]{list-style-
type:lower-roman}.mw-parser-output div.reflist-liststyle-upper-alpha
.references{list-style-type:upper-alpha}.mw-parser-output div.reflist-liststyle-
upper-roman .references{list-style-type:upper-roman}.mw-parser-output
div.reflist-liststyle-lower-alpha .references{list-style-type:lower-alpha}.mw-
parser-output div.reflist-liststyle-lower-greek .references{list-style-
type:lower-greek}.mw-parser-output div.reflist-liststyle-lower-roman
.references{list-style-type:lower-roman}\n^ Local time.\n\n^ For some countries
it is unclear whether they report samples or cases. One person tested twice is
recorded as one case and two samples.\n\n^ Excluding Taiwan.\n\n^ Excluding
Northern Cyprus.\n\n^ Excluding Greenland and the Faroe Islands.\n\n^ Excluding
Overseas France.\n\n^ Testing data from 4 May to 12 May is missing because of
the transition to the new reporting system SI-DEP.\n\n^ Excluding Abkhazia and
South Ossetia.\n\n^ Data for residents only.\n\n^ Excluding Transnistria.\n\n^
Northern Cyprus is not recognized as a sovereign state by any country except
Turkey.\n\n^ Includes data for Liechtenstein.\n\n^ Not a United Nations member."
[1] "United Arab Emirates"
```

```
[2] "United Kingdom"
[3] "United States"
[4] ".mw-parser-output .reflist-columns-2{column-width:30em}.mw-parser-output
.reflist-columns-3{column-width:25em}body.skin-vector-2022 .mw-parser-output
.reflist-columns-2{column-width:27em}body.skin-vector-2022 .mw-parser-output
.reflist-columns-3{column-width:22.5em}.mw-parser-output .references[data-mw-
group=upper-alpha]{list-style-type:upper-alpha}.mw-parser-output
.references[data-mw-group=upper-roman]{list-style-type:upper-roman}.mw-parser-
output .references[data-mw-group=lower-alpha]{list-style-type:lower-alpha}.mw-
parser-output .references[data-mw-group=lower-greek]{list-style-type:lower-
greek}.mw-parser-output .references[data-mw-group=lower-roman]{list-style-
type:lower-roman}.mw-parser-output div.reflist-liststyle-upper-alpha
.references{list-style-type:upper-alpha}.mw-parser-output div.reflist-liststyle-
upper-roman .references{list-style-type:upper-roman}.mw-parser-output
div.reflist-liststyle-lower-alpha .references{list-style-type:lower-alpha}.mw-
parser-output div.reflist-liststyle-lower-greek .references{list-style-
type:lower-greek}.mw-parser-output div.reflist-liststyle-lower-roman
.references{list-style-type:lower-roman}\n^ Local time.\n\n^ For some countries
it is unclear whether they report samples or cases. One person tested twice is
recorded as one case and two samples.\n\n^ Excluding Taiwan.\n\n^ Excluding
Northern Cyprus.\n\n^ Excluding Greenland and the Faroe Islands.\n\n^ Excluding
Overseas France.\n\n^ Testing data from 4 May to 12 May is missing because of
the transition to the new reporting system SI-DEP.\n\n^ Excluding Abkhazia and
South Ossetia.\n\n^ Data for residents only.\n\n^ Excluding Transnistria.\n\n^
Northern Cyprus is not recognized as a sovereign state by any country except
Turkey.\n\n^ Includes data for Liechtenstein.\n\n^ Not a United Nations member."
```

0.9 TASK 8: Pick two countries you are interested, and then review their testing data

The goal of task 8 is to compare the COVID-19 test data between two countries, you will need to select two rows from the dataframe, and select country, confirmed, confirmed-population-ratio columns

```
[18]: # Select a subset (should be only one row) of data frame based on a selected
      ↪country name and columns
selected_country <- "Canada"

country_subset <- subset(
  covid_data_frame_csv,
  country == selected_country,
  select = c(country, confirmed, tested)
)

print(country_subset)

# Select a subset (should be only one row) of data frame based on a selected
      ↪country name and columns
```

```

selected_country <- "United States"

country_subset <- subset(
  covid_data_frame_csv,
  country == selected_country,
  select = c(country,
             confirmed,
             tested,
             confirmed.population.ratio)
)

```

```

      country confirmed   tested
31 Canada           NA 66343123

```

0.10 TASK 9: Compare which one of the selected countries has a larger ratio of confirmed cases to population

The goal of task 9 is to find out which country you have selected before has larger ratio of confirmed cases to population, which may indicate that country has higher COVID-19 infection risk

```

[19]: # Use if-else statement
# if (check which confirmed.population value is greater) {
#   print()
# } else {
#   print()
# }

# Compare two countries by confirmed.population.ratio
# Select countries to compare
country1 <- "Canada"
country2 <- "United States"

# Extract confirmed.population.ratio values
ratio1 <- subset(
  covid_data_frame_csv,
  country == country1,
  select = confirmed.population.ratio
)$confirmed.population.ratio

ratio2 <- subset(
  covid_data_frame_csv,
  country == country2,
  select = confirmed.population.ratio
)$confirmed.population.ratio

# Use if-else statement
if (ratio1 > ratio2) {

```

```

print(
  paste(
    country1,
    "has a higher confirmed cases to population ratio than",
    country2
  )
)
} else if (ratio2 > ratio1) {

  print(
    paste(
      country2,
      "has a higher confirmed cases to population ratio than",
      country1
    )
  )
} else {

  print(
    paste(
      country1,
      "and",
      country2,
      "have the same confirmed cases to population ratio"
    )
  )
}

```

[1] "United States has a higher confirmed cases to population ratio than Canada"

0.11 TASK 10: Find countries with confirmed to population ratio rate less than a threshold

The goal of task 10 is to find out which countries have the confirmed to population ratio less than 1%, it may indicate the risk of those countries are relatively low

```

[20]: # Get a subset of any countries with `confirmed.population.ratio` less than the
      ↪ threshold
      # Define threshold (you can change this value as needed)
      threshold <- 0.1

      # Get subset of countries with confirmed.population.ratio less than threshold
      low_risk_countries <- subset(
        covid_data_frame_csv,

```

```

confirmed.population.ratio < threshold,
select = c(country,
            confirmed,
            tested,
            confirmed.population.ratio)
)

# Print result
print(low_risk_countries)

```

	country	confirmed	tested	confirmed.population.ratio
119	North Korea	NA	16914	0.0660
156	Tanzania	NA	3880	0.0065

[]: